ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Hoàng Thiện – MSV: 23020774

BÁO CÁO BÀI THỰC HÀNH 04
TURTLEBOT 4 SIMULATION CƠ BẢN

SLAM Toolbox – Localization (AMCL) – Navigation (Nav2)

Môn: [TÊN MÔN HỌC]
Ngành: Kỹ thuật Robot
Giảng viên hướng dẫn: Nguyễn Quốc Bảo

Hà Nội, tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày việc chạy TurtleBot 4 trong Gazebo (Ignition Fortress) trên ROS 2 Humble, dựng bản đồ bằng SLAM Toolbox, định vị trên bản đồ đã lưu bằng AMCL và điều hướng tự động bằng Nav2. Báo cáo mô tả cấu trúc chương trình, các node/topic chính, cách Nav2 lập kế hoạch và điều khiển robot, kèm kết quả đo thực tế và những hạn chế gặp phải khi chạy trên máy không có GPU phù hợp.

Mục lục

1 Tổng quan và môi trường thực hiện **2**
- 1.1 Mục tiêu . . . 2
- 1.2 Môi trường . . . 2
- 1.3 Những điều chỉnh so với cấu hình mặc định . . . 2

2 Cấu trúc chương trình **2**
- 2.1 Cấu trúc repository . . . 2
- 2.2 Các package ROS 2 được sử dụng . . . 3
- 2.3 Sơ đồ hệ thống . . . 3
- 2.4 Luồng chạy tổng thể . . . 3
- 2.5 Launch file chính . . . 3
- 2.6 World mô phỏng . . . 5

3 Phần 1 – TurtleBot 4 trong Gazebo **6**
- 3.1 Các bước thực hiện . . . 6
- 3.2 Kết quả kiểm tra topic . . . 6

4 Phần 2 – Xây dựng bản đồ bằng SLAM Toolbox **7**
- 4.1 Cách SLAM Toolbox hoạt động trong bài . . . 7
- 4.2 Chỉnh tham số cho mô phỏng chậm . . . 8
- 4.3 Điều khiển bằng teleoperation . . . 8
- 4.4 Lưu bản đồ . . . 9
- 4.5 Hậu xử lý bản đồ (cần nêu rõ) . . . 9
- 4.6 Đánh giá . . . 11

5 Phần 3 – Localization với bản đồ đã lưu **11**
- 5.1 Quy trình . . . 11
- 5.2 Tham số AMCL (`config/localization.yaml`) . . . 12
- 5.3 Đặt vị trí ban đầu . . . 12
- 5.4 Kết quả . . . 12

6 Phần 4 – Navigation với Nav2 **13**
- 6.1 Khởi chạy . . . 13
- 6.2 Nav2 lập kế hoạch và điều khiển như thế nào . . . 14
- 6.3 Thực hiện navigation goal . . . 15

7 Node, topic chính và trả lời câu hỏi **16**
- 7.1 Các node chính khi Nav2 đang chạy . . . 16
- 7.2 Các topic chính . . . 19
- 7.3 Trả lời các câu hỏi “Nội dung cần hiểu” . . . 19

8 Kết luận, hạn chế và liên kết **20**
- 8.1 Kết quả đạt được . . . 20
- 8.2 Hạn chế và những điều cần nói thẳng . . . 20
- 8.3 Liên kết . . . 21
- 8.4 Tài liệu tham khảo . . . 21

1 Tổng quan và môi trường thực hiện

1.1 Mục tiêu

Bài thực hành yêu cầu dùng TurtleBot 4 trong mô phỏng để (1) chạy robot trong Gazebo và kiểm tra `/scan`, `/odom`, `/cmd_vel`; (2) dựng bản đồ bằng SLAM Toolbox và lưu `map.yaml`, `map.pgm`; (3) nạp bản đồ và định vị bằng AMCL; (4) cho Nav2 điều hướng tự động tới ít nhất ba goal ở các khu vực khác nhau mà không điều khiển tay và không va chạm.

1.2 Môi trường

Hệ điều hành	Ubuntu 22.04 (Linux 6.8), cài trực tiếp
ROS 2	Humble Desktop; gói `turtlebot4_simulator`, `turtlebot4_navigation`, `nav2_bringup`
Mô phỏng	Ignition Gazebo 6 (Fortress), cầu nối `ros_gz_bridge`
Máy	Inspiron 3505, CPU AMD, GPU tích hợp Radeon Vega 8, RAM 14 GB
Thế giới	`worlds/maze_small.sdf`: sàn 8×8 m, có 2 vách ngăn và 2 hộp vật cản
Thời gian	dùng giờ mô phỏng (`use_sim_time:=true`) cho mọi node

Bảng 1: Môi trường thực hiện.

1.3 Những điều chỉnh so với cấu hình mặc định

Máy thực hành không render được LiDAR bằng GPU và khá chậm, nên có một số điều chỉnh. Đây là các thay đổi *ngoài* gói TurtleBot 4 gốc, nên được nêu rõ để người chấm tái hiện được:

- **Render bằng CPU**: đặt `LIBGL_ALWAYS_SOFTWARE=1`. Khi dùng GPU Radeon Vega 8, cảm biến LiDAR của Gazebo (ogre2) trả về toàn bộ 640 tia bằng đúng $0{,}164$ m (`range_min`), tức bản đồ không thể dựng được. Log kiểm tra: `logs/03_inspect_gz.log`.
- **World nhỏ hơn**: world `maze` gốc ($\approx 20 \times 20$ m) khiến mô phỏng chỉ chạy khoảng 6–16% thời gian thực. Em tự tạo world 8×8 m (sinh bằng `worlds/gen_world.py`).
- **Giảm tần số cảm biến** bằng thư mục `overlay/` (bản sao URDF đặt trước trong `AMENT_PREFIX_PATH`): LiDAR 62 Hz $\rightarrow$ 10 Hz, camera OAK-D 30 Hz $\rightarrow$ 1 Hz, cảm biến cliff/IR giảm còn 2 Hz. Nhờ đó (kèm tắt cửa sổ Gazebo khi không cần) hệ số thời gian thực lên khoảng 76%.
- **Mở khóa robot**: gói Create 3 dừng robot khi đi xa trạm sạc, nên cần đặt `safety_override=backup_only` và gọi action `/undock` (script `scripts/unlock_robot.sh`).

2 Cấu trúc chương trình

2.1 Cấu trúc repository

Repository chỉ chứa phần em viết thêm (launch, cấu hình, script, bản đồ); các gói TurtleBot 4, SLAM Toolbox và Nav2 được dùng nguyên bản từ ROS 2 Humble (`apt`).

```
tb4_slam_nav/
|-- launch/
|   |-- sim_small.launch.py      # Gazebo + TurtleBot 4 trong world nho (dung chinh)
|   `-- sim_maze.launch.py       # wrapper cho world maze goc cua TurtleBot 4
|-- worlds/  gen_world.py, maze_small.sdf     # world 8x8 m
|-- overlay/ turtlebot4_description, irobot_create_description   # URDF giam tan so cam
    bien
|-- config/  slam_small.yaml, localization.yaml, nav2.yaml
|-- maps/    map.yaml, map.pgm (+ map_raw.* la ban goc truoc khi loc)
|-- scripts/ start_sim.sh, unlock_robot.sh, start_slam.sh, teleop_slow.sh, save_map.sh,
|            clean_map.py, start_localization.sh, set_initial_pose.py, find_pose.py,
```

```
|            start_nav2.sh, send_goals.py, kill_sim.sh, check_*.{sh,py}, make_figures.py
|-- logs/    log terminal dung trong bao cao
|-- figures/ hinh dung trong bao cao
`-- report/  report.tex va cac phan (file nay)
```

Mã 1: Cây thư mục repository.

2.2 Các package ROS 2 được sử dụng

Package	Vai trò
`turtlebot4_ignition_bringup`	Spawn robot, dock, cầu nối Gazebo↔ROS (`/scan`, `/odom`, `/cmd_vel`, `/clock`, cảm biến)
`irobot_create_*`	Mô hình iRobot Create 3 (đế robot): `motion_control`, hazard, dock-/undock, `diffdrive_controller`
`turtlebot4_description`	URDF của robot, gắn RPLIDAR và camera OAK-D
`slam_toolbox`	Dựng bản đồ (node `async_slam_toolbox_node`)
`nav2_map_server`	`map_server` nạp `map.yaml`; `map_saver_cli` lưu bản đồ
`nav2_amcl`	Định vị Monte Carlo, phát phép biến đổi `map`→`odom`
`nav2_*` (bringup)	`bt_navigator`, `planner_server`, `controller_server`, `behavior_server`, `smoother_server`, `velocity_smoother`, hai costmap
`turtlebot4_navigation`	Launch/cấu hình sẵn: `slam.launch.py`, `localization.launch.py`, `nav2.launch.py`
`turtlebot4_viz`	Mở RViz2 với cấu hình có bảng điều khiển Nav2 và công cụ Goal
`teleop_twist_keyboard`	Lái robot bằng bàn phím khi quét bản đồ

Bảng 2: Các package chính.

2.3 Sơ đồ hệ thống

Hình 1 tóm tắt các node và luồng dữ liệu. Ba khối dưới cùng (mô phỏng, phần mềm robot) chạy suốt bài; khối giữa là SLAM Toolbox (Phần 2) hoặc `map_server`+AMCL (Phần 3), cả hai đều cung cấp TF `map`→`odom` nên không chạy đồng thời; khối trên là Nav2 (Phần 4) nhận goal từ RViz2 và phát `/cmd_vel` về robot.

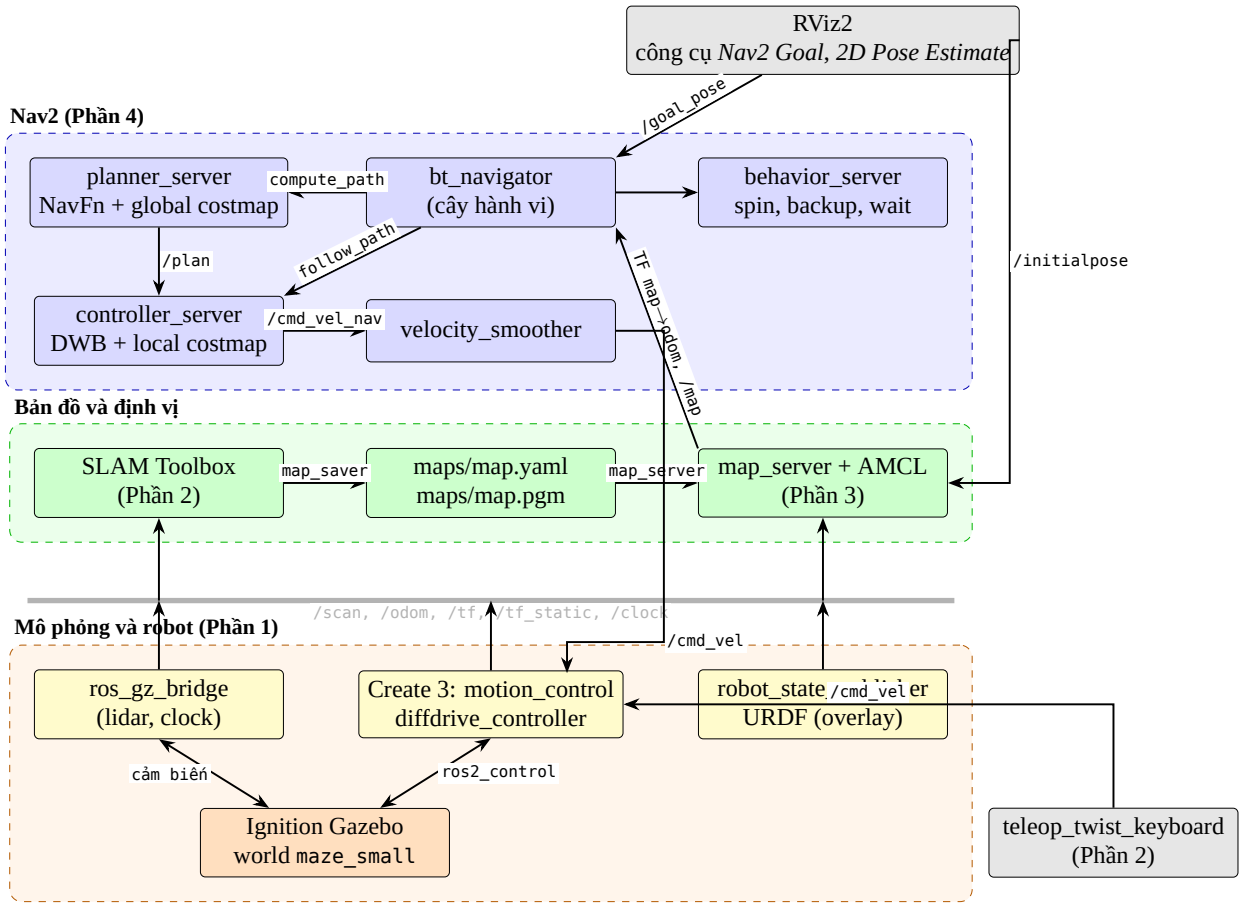
2.4 Luồng chạy tổng thể

1. `kill_sim.sh`: dọn mọi tiến trình Gazebo/ROS còn sót (tiến trình trùng là nguyên nhân gây lỗi `Frame [map] does not exist` và robot không chạy).
2. `start_sim.sh gui:=true`: khởi động Gazebo và robot; `unlock_robot.sh`: mở khóa và undock.
3. **Phần 2**: `start_slam.sh` + RViz2 + `teleop_slow.sh` → `save_map.sh` → `clean_map.py`.
4. **Phần 3**: tắt SLAM, khởi động lại sim, `start_localization.sh`, đặt initial pose.
5. **Phần 4**: `start_nav2.sh`, đặt goal bằng công cụ *Nav2 Goal* trong RViz2.

2.5 Launch file chính

Launch gốc `turtlebot4_ignition.launch.py` chỉ nạp world nằm trong thư mục của package, nên không dùng được world của repo. `launch/sim_small.launch.py` làm lại đúng các bước đó nhưng nạp world của repo:

```
WORLD_NAME = 'maze_small'
WORLD_FILE = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                          'worlds', WORLD_NAME + '.sdf')
```



Hình 1: Sơ đồ các node và luồng dữ liệu chính. Mũi tên lên từ robot là dữ liệu cảm biến (bus `/scan`, `/odom`, `/tf`); mũi tên xuống là lệnh vận tốc `/cmd_vel`. SLAM Toolbox (Phần 2) và AMCL (Phần 3) không chạy cùng lúc: cả hai đều phát TF `map`→`odom`.

```

def start_gazebo(context, *args, **kwargs):
    ...
    ign_args = f'{WORLD_FILE} -r -v 2'
    if gui:
        gui_config = os.path.join(pkg_bringup, 'gui', model, 'gui.config')
        ign_args += f' --gui-config {gui_config}'
    else:
        ign_args += ' -s'  # chi chay server, khong ve cua so (tiet kiem CPU)
    return [IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(pkg_ros_ign_gazebo, 'launch', '
    ign_gazebo.launch.py')),
        launch_arguments=[('ign_args', ign_args)])]
```

Mã 2: Trích `launch/sim_small.launch.py`: chạy Gazebo với world riêng.

Sau đó file này bắt cầu nối `/clock` và *include* `turtlebot4_spawn.launch.py` (có sẵn của `turtlebot4_ignition_bringup`) để spawn robot tại $(x, y) = (-2{,}5;\ -3{,}0)$. Script bao ngoài:

```
source /opt/ros/humble/setup.bash
export LIBGL_ALWAYS_SOFTWARE=1       # bat buoc: GPU AMD Vega 8 render LiDAR sai
export AMENT_PREFIX_PATH="$DIR/overlay/turtlebot4_description:$DIR/overlay/
    irobot_create_description:$AMENT_PREFIX_PATH"
exec ros2 launch "$DIR/launch/sim_small.launch.py" "$@"
```

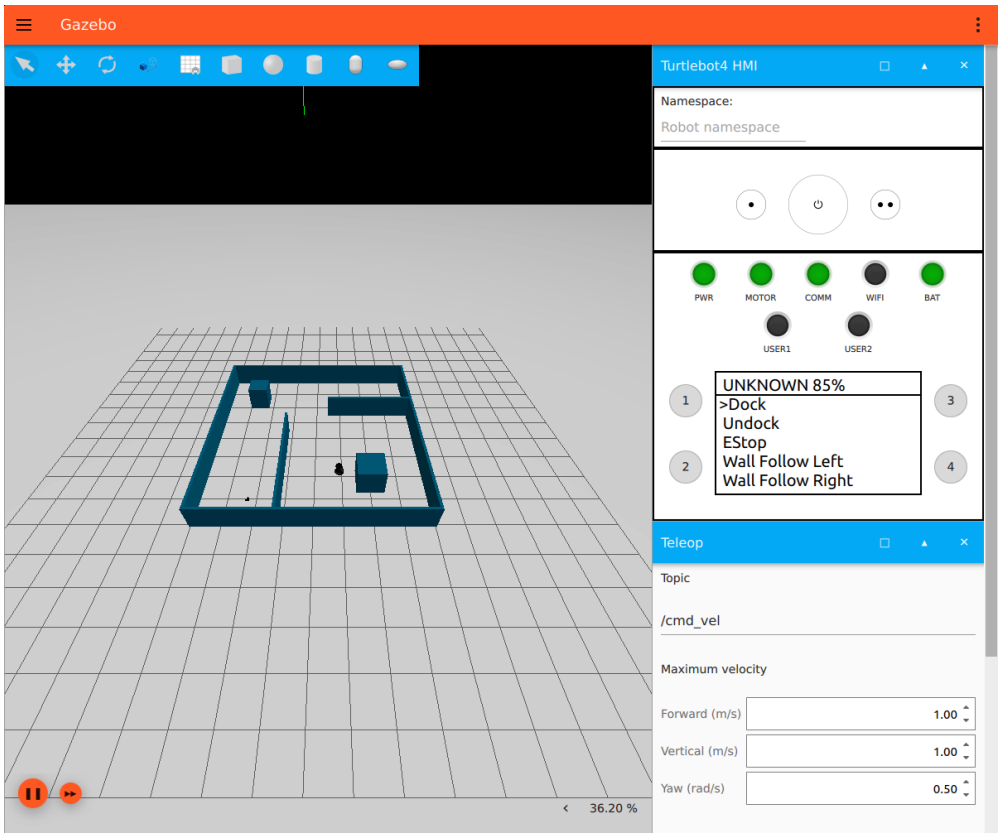
Mã 3: `scripts/start_sim.sh`.

2.6 World mô phỏng

`gen_world.py` sinh `maze_small.sdf`. Tường dày $0{,}1$ m, cao 1 m (LiDAR đặt khoảng $0{,}3$ m nên thấy được). Bố cục:

```
WALLS = [
    ('north', 0, 4, 8.1, T), ('south', 0, -4, 8.1, T),      # tuong bien 8x8 m
    ('east', 4, 0, T, 8.1),  ('west', -4, 0, T, 8.1),
    ('partition_a', -1.2, -1.75, T, 4.5),   # vach doc, chia san thanh 2 khu
    ('partition_b', 2.25, 1.5, 3.5, T),     # vach ngang tao hanh lang phia bac
    ('box_1', 2.0, -2.2, 1.0, 1.0),         # hai vat can hop
    ('box_2', -2.6, 2.4, 0.8, 0.8),
]
```

Mã 4: Danh sách tường trong `worlds/gen_world.py`.



Hình 2: Cửa sổ Gazebo nhìn từ trên: robot TurtleBot 4 (gần góc phải) trong world `maze_small` với hai vách ngăn và hai hộp vật cản.

3 Phần 1 – TurtleBot 4 trong Gazebo

3.1 Các bước thực hiện

```
scripts/kill_sim.sh                 # don tien trinh cu
scripts/start_sim.sh gui:=true      # Gazebo + robot (cua so Gazebo)
scripts/unlock_robot.sh             # safety_override=backup_only + /undock
source /opt/ros/humble/setup.bash
ros2 launch turtlebot4_viz view_robot.launch.py use_sim_time:=true    # RViz2
scripts/check_topics.sh             # ghi logs/01_topic_check.log
```

Script `unlock_robot.sh` chờ đến khi node `/motion_control` sẵn sàng rồi mới đặt tham số và undock:

```
until ros2 param get /motion_control safety_override >/dev/null 2>&1; do sleep 2; done
ros2 param set /motion_control safety_override backup_only
ros2 action send_goal /undock irobot_create_msgs/action/Undock "{}"
```

Mã 5: `scripts/unlock_robot.sh`.

Nếu gửi `/undock` khi Gazebo chưa lên hẳn thì goal bị *rejected*; chỉ cần chạy lại script.

3.2 Kết quả kiểm tra topic

Log thật từ `scripts/check_topics.sh` (rút gọn):

```
$ ros2 topic list | grep -E "^/(scan|odom|cmd_vel|tf|clock)$"
/clock  /cmd_vel  /odom  /scan  /tf

$ ros2 topic info /scan -v      -> sensor_msgs/msg/LaserScan,  publisher: lidar_bridge
$ ros2 topic info /odom -v      -> nav_msgs/msg/Odometry,      publisher: diffdrive_controller
```

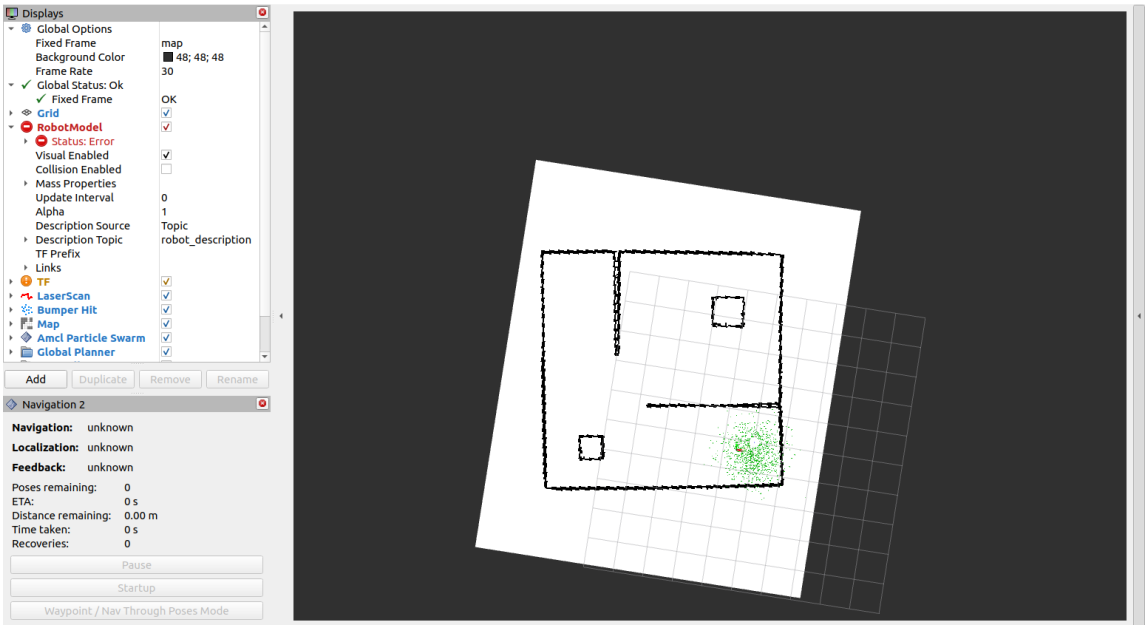
```
$ ros2 topic info /cmd_vel -v   -> geometry_msgs/msg/Twist,    publisher: cmd_vel_bridge

$ ros2 topic hz /scan   (6 s)   average rate: 2.250   (theo gio thuc cua may)
$ ros2 topic hz /odom   (6 s)   average rate: 21.410
$ ros2 topic echo /odom --once
header: {stamp: {sec: 1283}, frame_id: odom}      child_frame_id: base_link
```

Mã 6: Kiểm tra `/scan`, `/odom`, `/cmd_vel` (`logs/01_topic_check.log`).

- `/scan` do `lidar_bridge` (`ros_gz_bridge`) chuyển từ Gazebo sang ROS; mỗi bản tin có 640 tia, $-\pi \ldots \pi$, `range_max`=12 m.
- `/odom` do `diffdrive_controller` (ros2_control) phát từ encoder bánh xe mô phỏng, frame `odom`→`base_link`.
- `/cmd_vel` nhận `Twist`; node `motion_control` của Create 3 lọc lệnh rồi chuyển cho bộ điều khiển bánh. Khi đang có hành vi tự động (undock/dock), lệnh tay bị bỏ qua (log: *Ignoring velocities commanded while an autonomous behavior is running*).

Tần số trên là theo *giờ thực* của máy nên thấp hơn tần số trong mô phỏng (LiDAR đặt 10 Hz theo giờ mô phỏng). Hệ số thời gian thực đo được khoảng 76% khi chạy Gazebo không có GUI và thấp hơn khi mở GUI, vì máy phải render cảm biến bằng CPU.



Hình 3: RViz2 (cấu hình `turtlebot4_viz`, nhìn từ trên): mô hình robot, tia LiDAR (điểm đỏ) và các frame TF; bên trái là danh sách hiển thị (RobotModel, TF, LaserScan, Map, …).

4 Phần 2 – Xây dựng bản đồ bằng SLAM Toolbox

4.1 Cách SLAM Toolbox hoạt động trong bài

SLAM Toolbox nhận `/scan` và TF `odom`→`base_link`. Với mỗi scan đủ điều kiện (robot đã di chuyển quá ngưỡng), nó *scan matching* với các scan trước để hiệu chỉnh vị trí, thêm một nút vào đồ thị tư thế, rồi giải tối ưu đồ thị (Ceres) khi có *loop closure*. Kết quả là bản đồ lưới chiếm dụng trên `/map` và phép biến đổi `map`→`odom`. Launch dùng là `turtlebot4_navigation/slam.launch.py`, chỉ đổi file cấu hình và chọn node bất đồng bộ:

```
exec ros2 launch turtlebot4_navigation slam.launch.py use_sim_time:=true sync:=false \
  params:="$DIR/config/slam_small.yaml" "$@"
```

Mã 7: `scripts/start_slam.sh`.

4.2 Chỉnh tham số cho mô phỏng chậm

Với cấu hình mặc định, node đồng bộ (`sync:=true`) xử lý từng scan nên hàng đợi tăng (*Queue size has grown to: 40*), bản đồ cập nhật chậm và nhiễu. Em dùng node bất đồng bộ (`sync:=false`: bỏ qua scan khi đang bận) và chỉnh `config/slam_small.yaml` từ bản gốc `slam.yaml`:

Tham số	Gốc	Dùng
`throttle_scans`	1	2
`map_update_interval`	0.5	1.0
`max_laser_range`	12.0	10.0
`minimum_time_interval`	0.25	0.4
`transform_timeout`	0.2	0.5
`minimum_travel_distance`	0.0	0.05
`minimum_travel_heading`	0.0	0.1
`scan_buffer_maximum_scan_distance`	12.0	10.0

Bảng 3: Tham số SLAM Toolbox đã thay đổi so với `slam.yaml` gốc (các tham số còn lại giữ nguyên: Ceres, `resolution: 0.05`, `do_loop_closing: true`).

`minimum_travel_*` lớn hơn 0 giúp không thêm scan khi robot đứng yên (tránh nhiễu tích lũy); `transform_timeout` lớn hơn để chịu được TF đến muộn trên máy chậm.

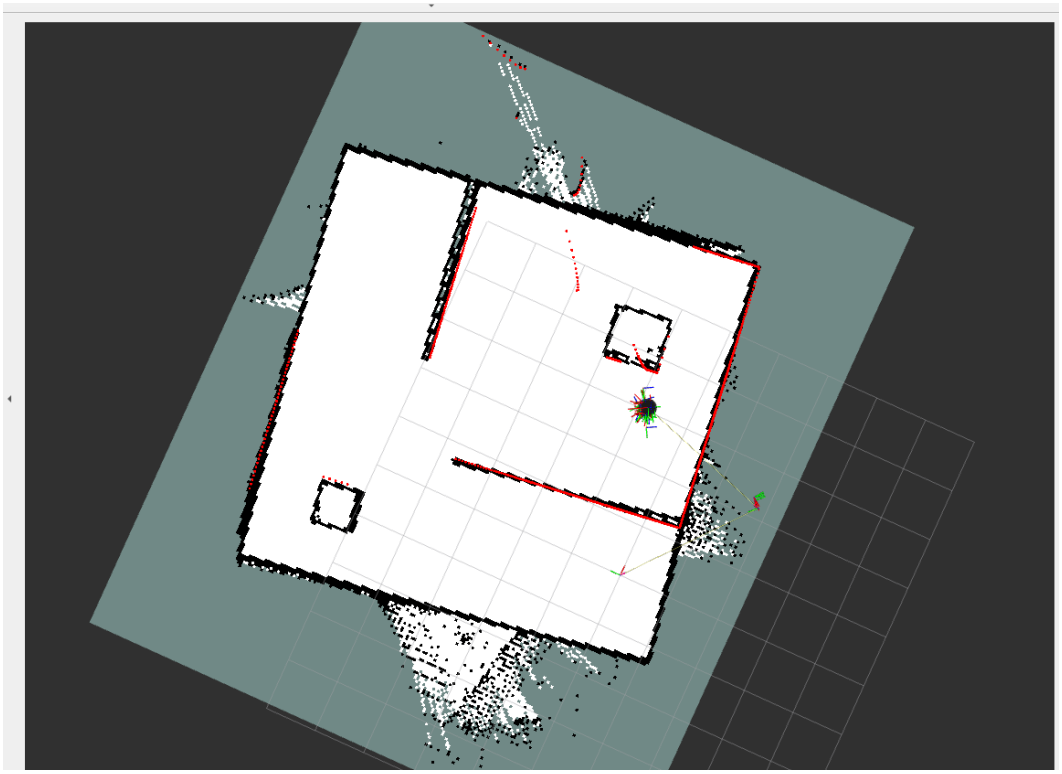
4.3 Điều khiển bằng teleoperation

Bản đồ được quét bằng `teleop_twist_keyboard` với tốc độ rất thấp để scan matching ổn định:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -p speed:="$SPEED" -p turn
    :="$TURN"
```

Mã 8: `scripts/teleop_slow.sh` (dùng 0,05 m/s và 0,2 rad/s).

Phím: `i` tiến, `,` lùi, `j`/`l` quay trái/phải, `k` dừng. Lái đi dọc các tường, vòng qua hai hộp vật cản và qua khe giữa hai khu để robot quét hết khu vực có thể di chuyển, đồng thời quan sát bản đồ hình thành trong RViz2. Em đã thử viết thêm một node tự động bám tường nhưng **không dùng** (không cho bản đồ tốt); bản đồ nộp là do lái tay.



Hình 4: Bản đồ đang hình thành trong RViz2 khi chạy SLAM Toolbox và lái robot bằng teleop: vùng xám chưa biết, vùng trắng đã quét, tia LiDAR màu đỏ trùng với tường.

4.4 Lưu bản đồ

```
ros2 run nav2_map_server map_saver_cli -f "$DIR/maps/map" --ros-args \
  -p save_map_timeout:=20.0 -p free_thresh_default:=0.25 -p occupied_thresh_default:=0.65
```

Mã 9: `scripts/save_map.sh` (rút gọn).

Kết quả là hai file `maps/map.pgm` (ảnh 262×220 điểm, 0,05 m/điểm) và `maps/map.yaml`:

```
image: map.pgm
mode: trinary
resolution: 0.05
origin: [-4.9, -2.32, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Mã 10: `maps/map.yaml`.

Ý nghĩa: `resolution` là kích thước một điểm ảnh (m); `origin` là tọa độ góc dưới-trái của ảnh trong frame `map`; điểm có xác suất chiếm dụng $> 0{,}65$ là tường (đen), $< 0{,}25$ là trống (trắng), còn lại là chưa biết (xám). Gốc `map` trùng vị trí robot ngay sau khi undock, vì SLAM được bật ở đó.

4.5 Hậu xử lý bản đồ (cần nêu rõ)

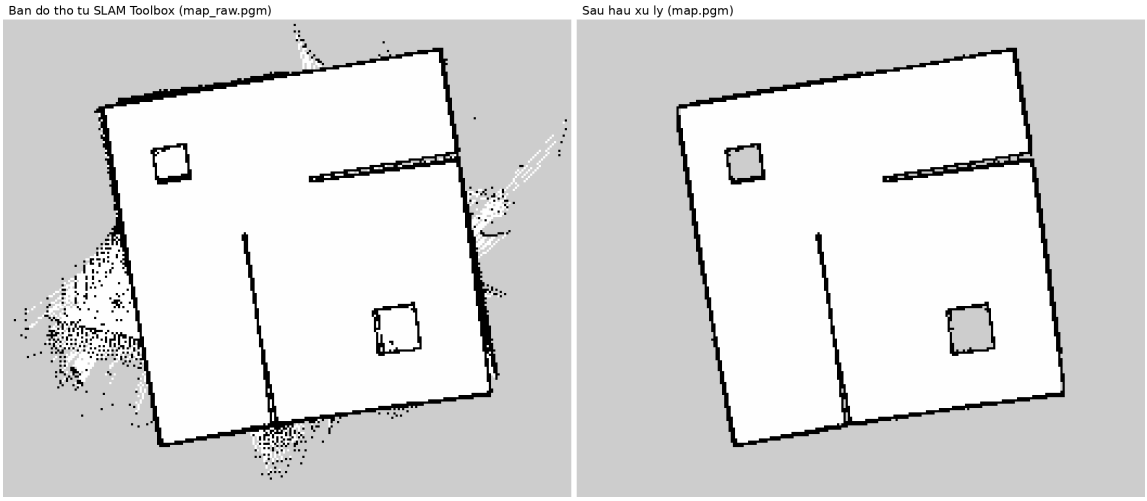
Bản đồ thô từ SLAM có nhiều điểm lẻ *bên ngoài* tường: tia LiDAR lọt qua khe hở nhỏ giữa các tấm tường ở góc và để lại các vệt trống/vật cản rời rạc. Đây không phải tường nhân đôi, nhưng làm bản đồ xấu và làm Nav2 thấy "vật cản ma". Em dùng `scripts/clean_map.py`: (i) co (erode) vùng trống bán kính 2 điểm rồi lấy thành phần liên thông lớn nhất, (ii) nở (dilate) trở lại trong vùng trống gốc, (iii) chỉ giữ ô tường sát vùng đó (≤ 2 điểm), phần còn lại đặt thành "chưa biết".

```
core = largest_component(erode(free, 2))
```

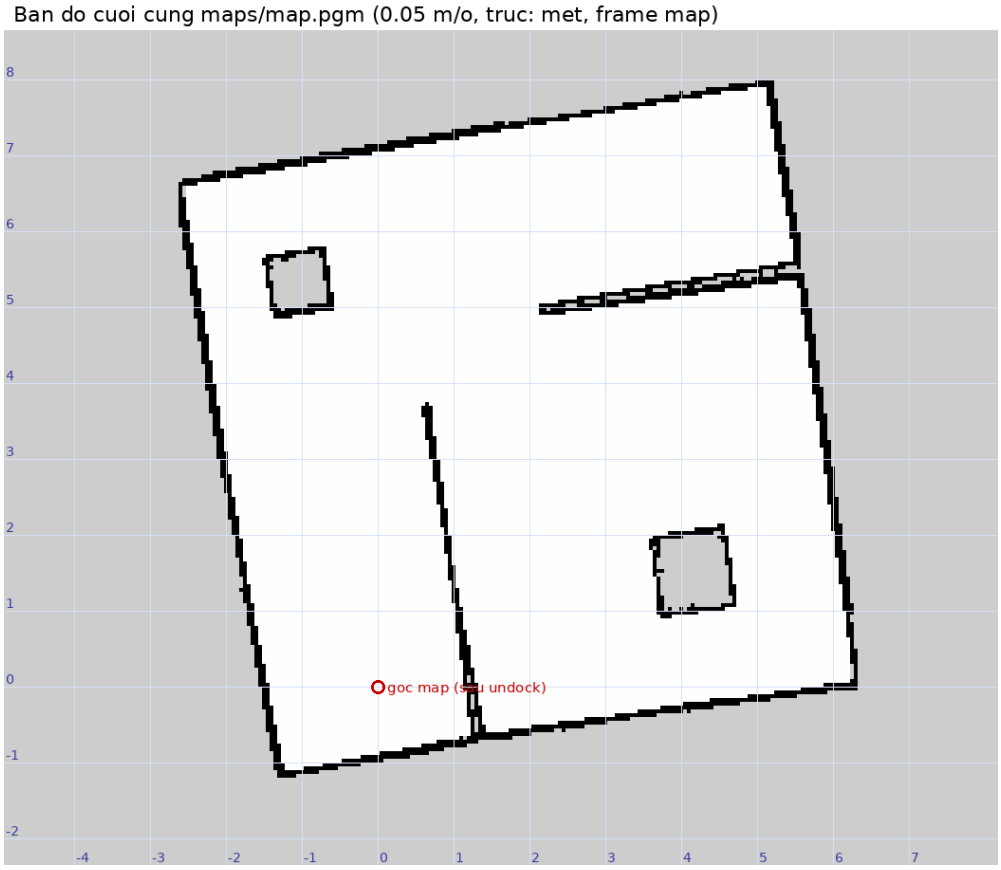
```
room = dilate(core, 2) & free                # vung free that su cua arena
keep_occ = occ & dilate(room, 2)             # tuong sat vung free
out = np.full_like(img, UNK)                 # UNK = 205 (xam)
out[room] = FREE                             # FREE = 254
out[keep_occ] = OCC                          # OCC = 0
```

Mã 11: Lõi của `scripts/clean_map.py`.

Kết quả: ô trống 25 412 → 23 048, ô tường 2 892 → 1 929. Cấu trúc bên trong (tường, vách, hai hộp) được giữ nguyên; bản gốc lưu ở `maps/map_raw.pgm`.



Hình 5: Bản đồ thô từ SLAM (trái) và sau hậu xử lý (phải).



Hình 6: Bản đồ cuối cùng `maps/map.pgm`. Trục tính bằng mét trong frame `map`; hai hộp vật cản hiện là khối xám vì LiDAR chỉ thấy mặt ngoài.

4.6 Đánh giá

Bản đồ có đủ tường biên, vách dọc, vách ngang (hành lang phía bắc) và hai hộp vật cản; không có tường nhân đôi hay vùng lệch rõ rệt. Toàn bộ bản đồ bị xoay nhẹ khoảng 5° so với hệ trục lưới, vì gốc `map` lấy theo hướng robot lúc bắt đầu quét, không phải theo trục của world. Điều này không ảnh hưởng đến localization và navigation vì chúng chỉ dùng chính bản đồ này.

5 Phần 3 – Localization với bản đồ đã lưu

5.1 Quy trình

Sau khi lưu bản đồ: dừng SLAM Toolbox, chạy `kill_sim.sh` rồi `start_sim.sh` + `unlock_robot.sh` để khởi động lại robot trong cùng world, sau đó chạy:

```
exec ros2 launch turtlebot4_navigation localization.launch.py use_sim_time:=true \
  map:="$DIR/maps/map.yaml" params:="$DIR/config/localization.yaml" "$@"
```

Mã 12: `scripts/start_localization.sh`.

Launch này gọi `nav2_bringup/localization_launch.py`, khởi động `map_server` (nạp `maps/map.yaml`, phát `/map`) và `amcl`, được quản lý bởi `lifecycle_manager_localization`. Log xác nhận cả hai node ở trạng thái *active*:

```
[lifecycle_manager_localization]: Server amcl connected with bond.
[lifecycle_manager_localization]: Managed nodes are active
[amcl]: AMCL cannot publish a pose or update the transform. Please set the initial pose...
```

Mã 13: Log `start_localization.sh` (rút gọn).

Dòng cuối là bình thường: AMCL cần vị trí ban đầu mới bắt đầu phát `/amcl_pose`.

5.2 Tham số AMCL (`config/localization.yaml`)

Giữ nguyên cấu hình của `turtlebot4_navigation`: mô hình chuyển động vi sai (`DifferentialMotionModel`), mô hình cảm biến `likelihood_field`, `min_particles`=500, `max_particles`=2000, `update_min_d`=0,25 m, `update_min_a`=0,2 rad, `global_frame_id: map`, `odom_frame_id: odom`, `base_frame_id: base_link`, `tf_broadcast: true`. Nghĩa là AMCL chỉ cập nhật khi robot đi quá 0,25 m hoặc quay quá 0,2 rad, và bù lệch bằng cách phát TF `map`→`odom`.

5.3 Đặt vị trí ban đầu

AMCL cần `/initialpose` (tương đương công cụ *2D Pose Estimate* trong RViz2). Em dùng script publish `PoseWithCovarianceStamped`:

```
msg.header.frame_id = 'map'
msg.pose.pose.position.x = x
msg.pose.pose.position.y = y
msg.pose.pose.orientation.z = math.sin(yaw / 2.0)
msg.pose.pose.orientation.w = math.cos(yaw / 2.0)
cov[0] = cov[7] = sxy ** 2      # phuong sai vi tri (mac dinh sigma 0.5 m)
cov[35] = syaw ** 2             # phuong sai huong (mac dinh sigma 0.26 rad ~ 15 do)
```

Mã 14: Trích `scripts/set_initial_pose.py`.

Hiệp phương sai càng nhỏ thì các hạt (particle) càng tập trung quanh vị trí cung cấp.

Vì Gazebo ở máy này không cung cấp được vị trí thật (topic `/sim_ground_truth_pose` xung đột QoS), em tự tìm vị trí robot từ bản đồ bằng `scripts/find_pose.py`: lấy một bản tin `/scan`, thử lưới các tư thế (x, y, θ) trên bản đồ và chọn tư thế làm các điểm scan rơi sát tường nhất (sai số RMS đến ô tường gần nhất, dùng distance transform):

```
gx, gy = x + c * lx - sn * ly, y + sn * lx + c * ly      # diem scan -> frame map
ix = ((gx - OX) / RES).astype(int);  iy = H - 1 - ((gy - OY) / RES).astype(int)
e = np.sqrt(np.mean(dist[iy[m], ix[m]] ** 2))            # RMS khoang cach toi tuong gan
    nhat
```

Mã 15: Trích `scripts/find_pose.py`.

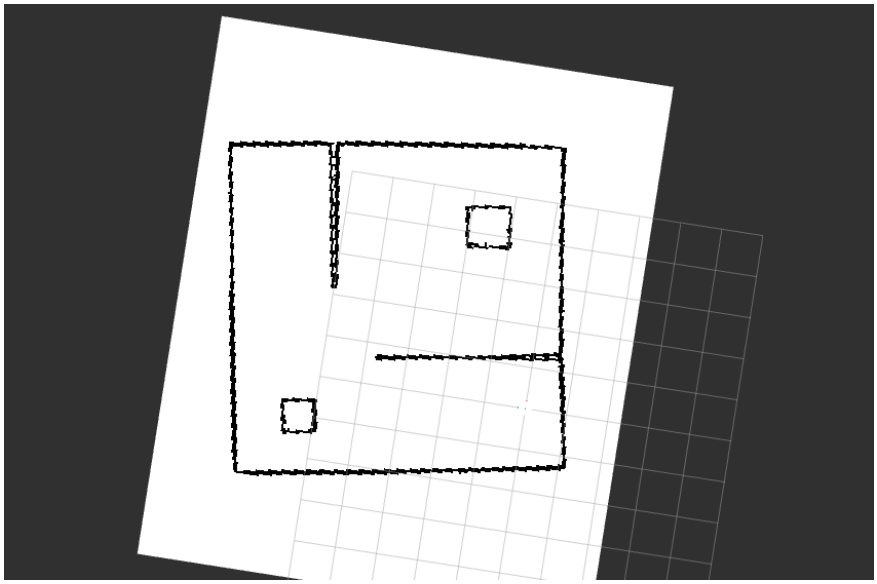
5.4 Kết quả

```
$ python3 scripts/find_pose.py -0.8 0.4 -0.6 0.4
best -0.10 -0.17 2.862 ...                    # x, y, yaw(rad) khop voi ban do
$ python3 scripts/set_initial_pose.py -0.10 -0.17 2.862 0.15 0.15
Da dat initial pose: x=-0.1 y=-0.17 yaw=2.862 rad (frame map)
[amcl]: Setting pose: -0.100 -0.170 2.862
$ python3 scripts/check_localization.py
AMCL  : x=-0.089 y=-0.174 yaw=164.1 deg
        do bat dinh: sigma_x=0.155 m  sigma_y=0.152 m  sigma_yaw=8.3 deg
```

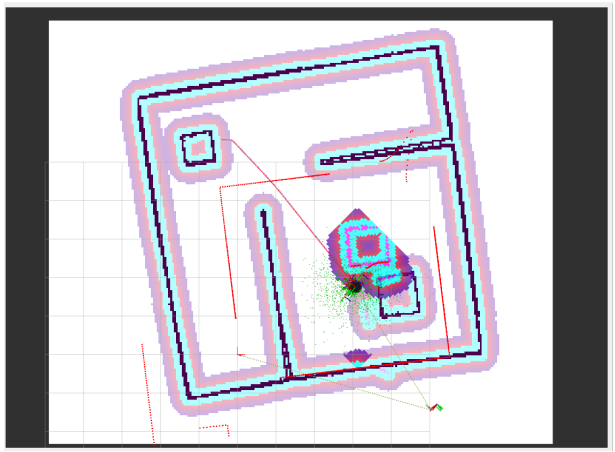
Mã 16: Đặt initial pose và kiểm tra AMCL (`scripts/check_localization.py`).

AMCL báo $(-0{,}089;\ -0{,}174;\ 164{,}1^\circ)$, trùng với kết quả khớp scan–bản đồ $(-0{,}10;\ -0{,}17;\ 164{,}0^\circ)$, độ bất định còn khoảng 15 cm và 8°. Cần lưu ý đây **không phải một phép kiểm chứng độc lập**, vì chính kết quả khớp scan đã được dùng làm initial pose; nó chỉ cho thấy AMCL giữ được tư thế đó. Kiểm chứng thực tế là: tia LiDAR trong RViz2 phải trùng các tường của bản đồ và đám hạt phải tụ gọn quanh robot (hình bên dưới), và trong Phần 4 robot đi đến đúng các điểm trên bản đồ.

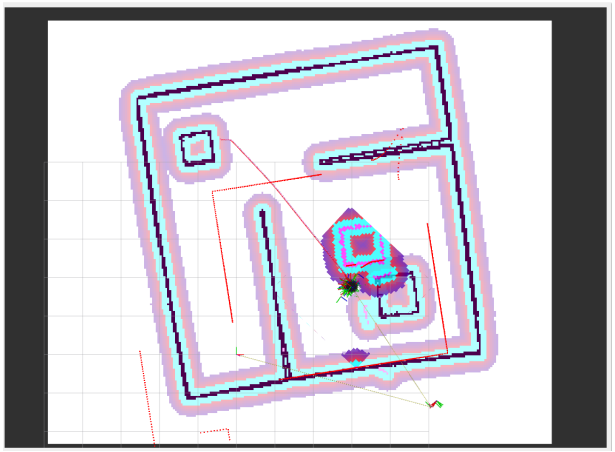
Lưu ý khi đặt initial pose. Các lần đầu em đặt initial pose theo ước lượng bằng mắt (ví dụ $(0, 0, 0)$ hoặc theo `/odom`). Sau khi robot di chuyển, AMCL vẫn ở tư thế lệch: tại một thời điểm, tư thế AMCL cho sai số khớp scan–bản đồ 0,34 m, trong khi tư thế tìm bằng `find_pose.py` chỉ 0,09 m. AMCL chỉ cập nhật khi robot đi đủ xa và cần initial pose đủ gần mới hội tụ đúng, nên em chuyển sang dùng `find_pose.py` để lấy initial pose. Em chưa xác định chắc nguyên nhân gốc của sai lệch ban đầu (nghi do vị trí/hướng robot sau undock khác giả định, và sim chạy chậm làm TF đến trễ).



Hình 7: RViz2 sau khi `map_server` nạp lại `maps/map.yaml`: bản đồ đã lưu hiện lên trên topic `/map` (chưa đặt initial pose nên chưa có robot trên bản đồ).



(a) initial pose với $\sigma_{xy} = 0{,}5\,\text{m}$, $\sigma_\theta = 0{,}4\,\text{rad}$: hạt (chấm xanh lá) rải rộng.



(b) với $\sigma_{xy} = 0{,}1\,\text{m}$, $\sigma_\theta = 0{,}1\,\text{rad}$: hạt tụ gọn quanh robot; AMCL báo $\sigma \approx 0{,}10\,\text{m}$, $5{,}4^\circ$.

Hình 8: Đám hạt AMCL trong RViz2 (vùng màu quanh robot là local costmap; tia LiDAR đỏ trùng tường bản đồ).

6 Phần 4 – Navigation với Nav2

6.1 Khởi chạy

Nav2 chạy *sau* localization (AMCL cung cấp TF `map`→`odom` và `map_server` cung cấp `/map`) và sau khi đã đặt initial pose:

```
exec ros2 launch turtlebot4_navigation nav2.launch.py use_sim_time:=true \
  params_file:="$DIR/config/nav2.yaml" "$@"
```

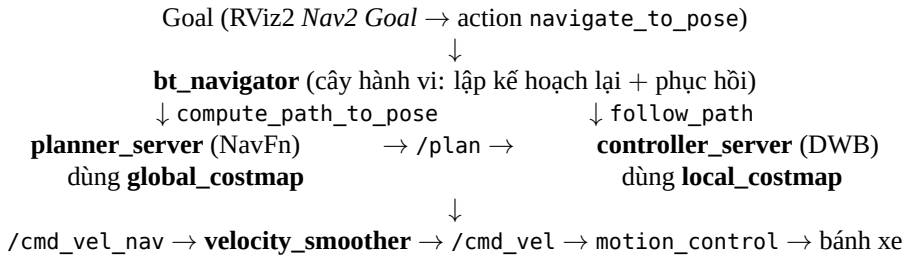
Mã 17: `scripts/start_nav2.sh`.

Launch này gọi `nav2_bringup/navigation_launch.py`, đặt lại tên `cmd_vel`: controller phát `cmd_vel_nav`, `velocity_smoother` làm mượt rồi phát `cmd_vel` (topic mà `motion_control` của robot nhận). Log xác nhận các node đã *active*:

```
[lifecycle_manager_navigation]: Server velocity_smoother connected with bond.
[lifecycle_manager_navigation]: Managed nodes are active
```

Mã 18: Cuối log `start_nav2.sh`.

6.2 Nav2 lập kế hoạch và điều khiển như thế nào



Hình 9: Luồng dữ liệu của Nav2 trong bài.

Bản đồ chi phí (costmap). `global_costmap` (frame `map`, cập nhật 1 Hz) gồm `static_layer` (từ `/map`), `obstacle_layer` (từ `/scan`) và `inflation_layer`. `local_costmap` (frame `odom`, cửa sổ trượt 3×3 m, 5 Hz) dùng `voxel_layer`. `inflation_layer` làm phồng vật cản: các ô trong bán kính `inflation_radius` quanh tường có chi phí giảm dần theo $\exp(-\text{cost_scaling_factor} \cdot (d - r_{\text{robot}}))$, nên đường đi tránh sát tường.

```
robot_radius: 0.175
inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  cost_scaling_factor: 4.0
  inflation_radius: 0.45
obstacle_layer / voxel_layer:
  observation_sources: scan
  raytrace_max_range: 3.0
  obstacle_max_range: 2.5
```

Mã 19: Trích `config/nav2.yaml`: costmap.

Lập kế hoạch toàn cục (planner_server). Dùng `nav2_navfn_planner/NavfnPlanner` (Dijkstra, `use_astar: false`) trên `global_costmap`; `tolerance: 0.5` cho phép goal lệch tối đa 0,5 m nếu ô goal không đi được. Kết quả là đường đi ngắn nhất tránh vùng chi phí cao, phát trên `/plan`.

```
planner_server:
  ros__parameters:
    expected_planner_frequency: 20.0
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
```

Mã 20: Trích `config/nav2.yaml`: planner.

Điều khiển cục bộ (controller_server). Chạy ở 20 Hz với `dwb_core::DWBLocalPlanner` (Dynamic Window Approach). Mỗi chu kỳ DWB lấy mẫu các cặp (v, ω) khả dĩ trong cửa sổ động (giới hạn tốc độ và gia tốc), mô phỏng quỹ đạo `sim_time`=1,7 s về phía trước trên `local_costmap`, chấm điểm từng quỹ đạo bằng các *critic* và chọn quỹ đạo điểm tốt nhất: `BaseObstacle` (tránh vật cản), `PathAlign`/`PathDist` (bám đường toàn cục), `GoalAlign`/`GoalDist` (tiến gần goal), `RotateToGoal` (xoay đúng hướng ở đích), `Oscillation` (chống dao động).

```
controller_server:
  ros__parameters:
    controller_frequency: 20.0
    general_goal_checker:
      plugin: "nav2_controller::SimpleGoalChecker"
      xy_goal_tolerance: 0.25
      yaw_goal_tolerance: 0.25
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      max_vel_x: 0.26      max_vel_theta: 1.0
      acc_lim_x: 2.5       acc_lim_theta: 3.2
      vx_samples: 20       sim_time: 1.7
      critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign",
                "PathAlign", "PathDist", "GoalDist"]
```

Mã 21: Trích `config/nav2.yaml`: controller.

Cây hành vi và phục hồi (bt_navigator, behavior_server). Mặc định dùng cây `navigate_to_pose_w_replanning_and_recovery`: lập kế hoạch lại định kỳ, theo đường đi; nếu thất bại thì xóa costmap và chạy các hành vi phục hồi của `behavior_server` (`spin`, `backup`, `wait`, ...) rồi thử lại. Goal chỉ "thành công" khi robot vào trong `xy_goal_tolerance`=0,25 m (và đúng hướng trong 0,25 rad).

Làm mượt vận tốc. `velocity_smoother` giới hạn tốc độ $\pm 0{,}26$ m/s, $\pm 1{,}0$ rad/s và gia tốc $2{,}5$ m/s^2, $3{,}2$ rad/s^2 trước khi gửi `/cmd_vel`.

6.3 Thực hiện navigation goal

Sau khi AMCL đã hội tụ (Phần 3), em đặt goal. Trong video demo, goal được đặt bằng công cụ *Nav2 Goal* của RViz2 (không lái tay sau đó). Để có số liệu có thể kiểm chứng (thời gian, quãng đường, khoảng cách tối thiểu tới tường), em còn dùng `scripts/send_goals.py`, gửi cùng loại goal `NavigateToPose` qua action server và ghi lại vị trí AMCL và khoảng cách nhỏ nhất của `/scan`:

```
client = ActionClient(node, NavigateToPose, 'navigate_to_pose')
msg = NavigateToPose.Goal()
msg.pose.header.frame_id = 'map'
msg.pose.pose.position.x, msg.pose.pose.position.y = gx, gy
msg.pose.pose.orientation.z, msg.pose.pose.orientation.w = math.sin(gyaw/2), math.cos(gyaw
    /2)
fut = client.send_goal_async(msg)
```

Mã 22: Trích `scripts/send_goals.py`.

Bốn goal ở bốn khu vực khác nhau của bản đồ (tọa độ trong frame `map`):

Goal	Khu vực	Tọa độ	Kết quả	Quãng đường	Cách tường gần nhất
1	phải-giữa, gần hộp	(3,0; 2,0)	thành công (lần 2)	1,62 m	0,37 m
2	hành lang phía bắc	(2,5; 5,6)	thành công	7,41 m	0,46 m
3	góc tây-bắc, cạnh hộp	$(-0{,}4;\ 5{,}6)$	thành công	2,60 m	0,48 m
4	đông-nam	(3,5; 0,2)	thành công	6,73 m	0,41 m

Bảng 4: Kết quả bốn goal (nguồn: `logs/04_nav_goals.log`, `logs/05_nav_goal1_retry.log`). Sai số tới đích đo bằng `/amcl_pose` sau khi goal kết thúc: 0,21; 0,20; 0,28; 0,19 m (goal 1–4). Goal checker xác nhận "thành công" với ngưỡng 0,25 m; số 0,28 m của goal 3 hơi lớn hơn ngưỡng. Em chưa xác định chắc lý do; có thể do `/amcl_pose` chỉ cập nhật khi robot đi đủ `update_min_d`=0,25 m nên giá trị em đọc có thể cũ, hoặc do sai số định vị AMCL, nên không nên coi con số này chính xác hơn vài cm.

Về goal 1. Lần đầu goal 1 bị *ABORTED*: planner báo *"Planning algorithm GridBased failed to generate a valid path to (3.00, 2.00)"*, robot chỉ đi 0,85 m. Đến khi chạy lại goal này (sau goal 2–4) thì thành công trong 39 s. Em chưa xác định chắc nguyên nhân thất bại lần đầu; thời điểm đó AMCL vừa hội tụ và robot ở gần vị trí xuất phát, nên có thể do costmap/định vị chưa ổn định hoặc robot đứng trong vùng phồng của vách. Vì thế 3 goal liên tiếp đầu tiên đạt yêu cầu là goal 2, 3, 4; goal 1 thành công ở lần thử lại.

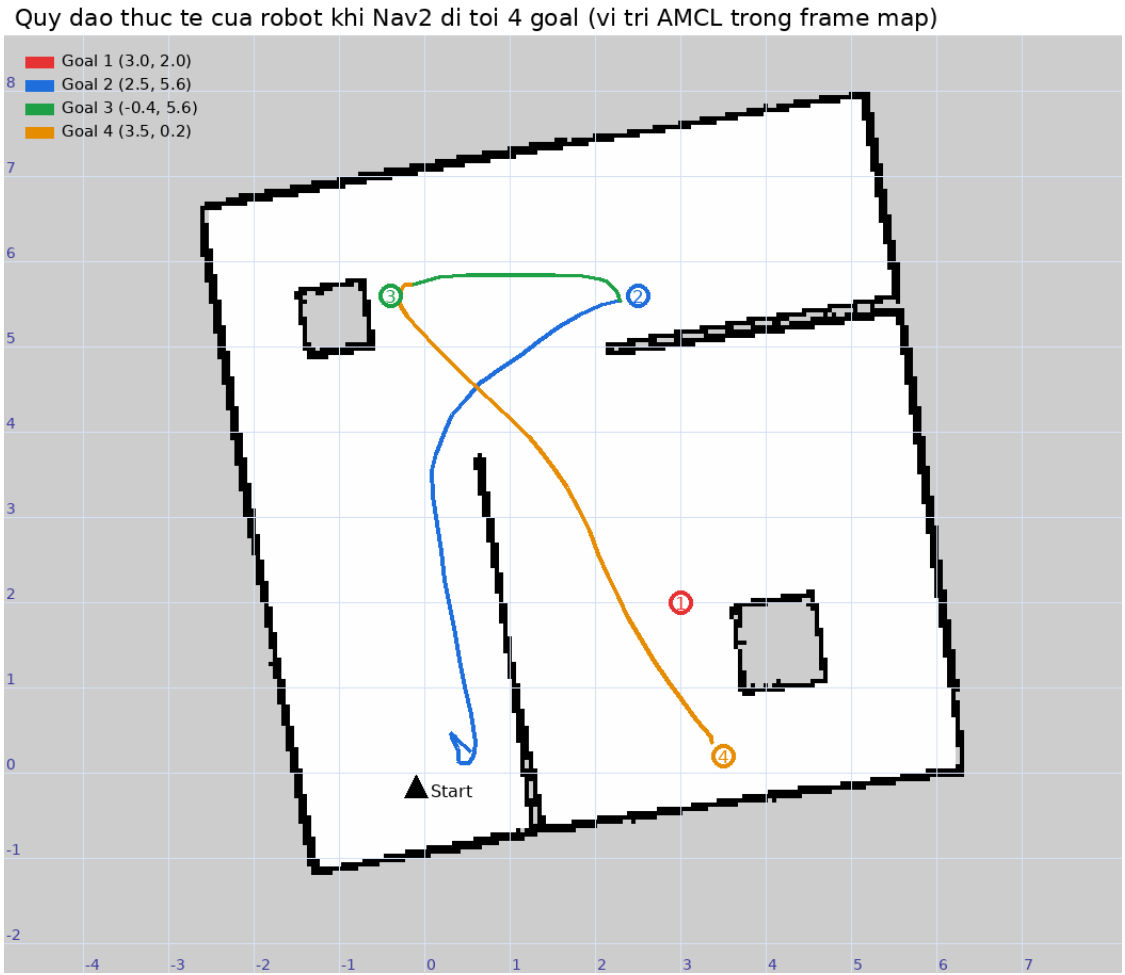
Va chạm. Khoảng cách nhỏ nhất từ LiDAR tới vật cản trong cả hành trình là 0,37–0,48 m (robot bán kính 0,175 m), tức không sát hay chạm tường. Số liệu này lấy từ `/scan` nên chỉ phản ánh vật cản trong mặt phẳng LiDAR. Log của `motion_control` chỉ có các dòng bật reflex lúc khởi động (*Enabling REFLEX_BUMP/CLIFF/STUCK/WHEEL_DROP*), không có sự kiện hazard hay reflex nào trong lúc chạy Nav2; em không ghi riêng topic `/hazard_detection`. Quỹ đạo thật vẽ ở hình dưới.

Lần gửi goal thứ 5 (thất bại). Để chụp Hình 11, sau khi hệ thống đã chạy liên tục hơn 2 200 s giờ mô phỏng, em gửi lại goal $(-0{,}4;\ 5{,}6)$. Planner vẫn tạo được đường đi (Hình 11) nhưng robot không tiến được: `controller_server` liên tục báo *"Transform data too old when converting from map to odom"* (TF `map`→`odom` mà nó giữ có mốc thời gian 1 235 s trong khi dữ liệu là 2 247 s), rồi *"Failed to make progress"*; hành vi phục hồi `spin` cũng thất bại vì *"Current footprint not available"*. Script hủy goal sau 600 s; robot chỉ dao động quanh chỗ cũ (quãng đường tích lũy 10,4 m nhưng vị trí thay đổi dưới 0,25 m) và không va chạm (cách tường $\geq 0{,}50$ m). Em chưa xác định được nguyên nhân gốc; nghi là bộ đệm TF của Nav2 bị lệch sau một lần RViz2 bị tắt và mở lại, kết hợp máy quá tải (load average 48–60). Vì vậy số liệu chính thức của Phần 4 là 4 goal ở Bảng 4; lần thứ 5 được ghi lại để trung thực (`logs/07_nav_goal_rviz_shot.log`).

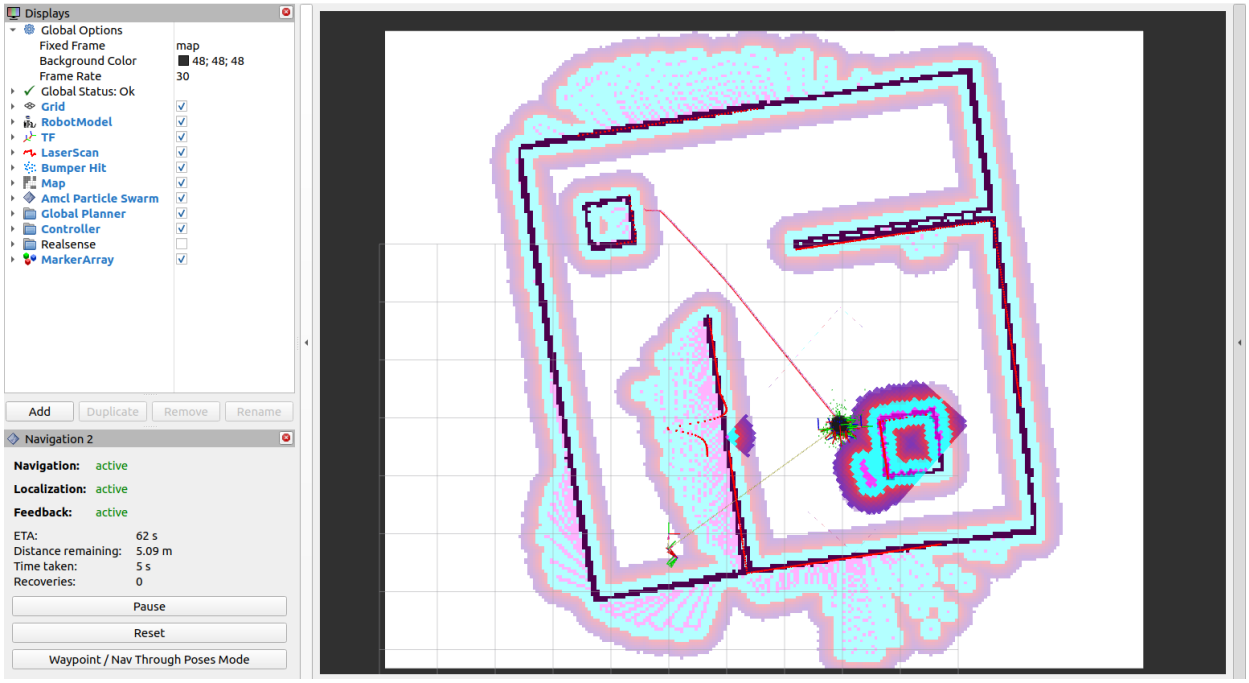
7 Node, topic chính và trả lời câu hỏi

7.1 Các node chính khi Nav2 đang chạy

Danh sách thu từ `ros2 node list` (`logs/06_system_info.log`), lọc các node liên quan:



Hình 10: Quỹ đạo robot (vị trí AMCL) khi Nav2 đi đến goal 2, 3, 4, vẽ trên bản đồ từ `logs/nav_trajectory_run1.csv`. Số trong vòng tròn là vị trí goal, tam giác đen là vị trí xuất phát. Quỹ đạo của lần chạy lại goal 1 không vẽ được vì file CSV của lần đó đã bị lần gửi goal thứ 5 ghi đè (script lúc đó luôn ghi cùng tên file); chỉ còn số liệu tổng hợp trong `logs/05_nav_goal1_retry.log`.



Hình 11: RViz2 khi Nav2 đang thực hiện một goal: global costmap (vùng màu quanh tường là inflation), local costmap (ô vuông quanh robot), đường đi toàn cục màu đỏ từ robot tới goal ở góc tây-bắc, hạt AMCL màu xanh lá; bảng *Navigation 2* bên trái báo Navigation/Localization: active. Ảnh chụp ở lần gửi goal thứ 5 (xem mục 6.3).

Node	Chức năng
`/lidar_bridge`	Cầu nối Gazebo → ROS cho `/scan`
`/diffdrive_controller`	Điều khiển bánh vi sai, phát `/odom` và TF `odom`→`base_link`
`/motion_control`	Create 3: nhận `/cmd_vel`, áp dụng an toàn (reflex, `safety_override`)
`/robot_state_publisher`	Phát TF tĩnh từ URDF (`base_link`→`rplidar_link`, ...)
`/map_server`	Nạp `map.yaml`, phát `/map`
`/amcl`	Định vị, nhận `/scan`, `/odom`, `/initialpose`; phát `/amcl_pose`, `/particle_cloud`, TF `map`→`odom`
`/bt_navigator`	Máy chủ action `navigate_to_pose`; chạy cây hành vi
`/planner_server`	Lập kế hoạch toàn cục (NavFn), action `compute_path_to_pose`
`/controller_server`	Điều khiển cục bộ (DWB), action `follow_path`
`/global_costmap`, `/local_costmap`	Hai bản đồ chi phí
`/behavior_server`	Hành vi phục hồi: `spin`, `backup`, `wait`
`/smoother_server`, `/velocity_smoother`	Làm mượt đường đi và vận tốc
`/lifecycle_manager_*`	Khởi động/giám sát các node vòng đời (lifecycle)

Bảng 5: Node chính.

7.2 Các topic chính

Topic	Kiểu	Ghi chú
`/scan`	`sensor_msgs/LaserScan`	LiDAR, SLAM/AMCL/costmap đều d
`/odom`	`nav_msgs/Odometry`	odometry bánh xe
`/cmd_vel`	`geometry_msgs/Twist`	lệnh vận tốc tới robot
`/cmd_vel_nav`	`geometry_msgs/Twist`	đầu ra controller, trước smoother
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	cây TF
`/clock`	`rosgraph_msgs/Clock`	giờ mô phỏng
`/map`	`nav_msgs/OccupancyGrid`	bản đồ (SLAM hoặc `map_server`)
`/initialpose`	`PoseWithCovarianceStamped`	vị trí ban đầu cho AMCL
`/amcl_pose`	`PoseWithCovarianceStamped`	kết quả định vị
`/particle_cloud`	`nav2_msgs/ParticleCloud`	các hạt AMCL
`/goal_pose`	`geometry_msgs/PoseStamped`	goal từ RViz2
`/plan`, `/plan_smoothed`	`nav_msgs/Path`	đường đi toàn cục
`/global_costmap/costmap`, `/local_costmap/costmap`	`nav_msgs/OccupancyGrid`	costmap

Bảng 6: Topic chính.

Số publisher của `/cmd_vel` đo được là 7 vì nhiều node (teleop, smoother, bridge, ...) cùng có thể phát; chỉ `velocity_smoother` phát khi Nav2 điều khiển.

Cây TF. Chuỗi là `map` $\xrightarrow{\text{AMCL}}$ `odom` $\xrightarrow{\text{diffdrive}}$ `base_link` $\xrightarrow{\text{URDF}}$ `rplidar_link`. `odom`→`base_link` liên tục nhưng trôi theo thời gian; `map`→`odom` do AMCL (hoặc SLAM) bù để đưa về khung bản đồ cố định. Ví dụ đo được: `tf2_echo map base_link` cho $(3{,}055;\ 1{,}801)$, yaw $13{,}5°$ khi robot đang ở gần goal 1.

7.3 Trả lời các câu hỏi "Nội dung cần hiểu"

Ghi chú: đề bài không kèm danh sách câu hỏi cụ thể trong bản em có, nên dưới đây là các câu hỏi cốt lõi về chủ đề bài; nếu đề có câu khác, em sẽ bổ sung.

1. SLAM là gì, SLAM Toolbox làm gì? SLAM (Simultaneous Localization and Mapping) là dựng bản đồ và ước lượng vị trí robot cùng lúc khi chưa biết trước cả hai. SLAM Toolbox là SLAM 2D dựa trên đồ thị tư thế: mỗi scan là một nút, scan matching tạo ràng buộc giữa các nút kề nhau, loop closure tạo ràng buộc giữa các nút xa nhau khi quay lại chỗ cũ, rồi bộ giải (Ceres) tối ưu toàn đồ thị để giảm lệch tích lũy.

2. Vì sao cần cả `/odom` lẫn `/scan`? `/odom` cho ước lượng chuyển động ngắn hạn, liên tục và mượt nhưng trôi (trượt bánh, sai số tích lũy). `/scan` đo khoảng cách tuyệt đối tới môi trường, dùng để sửa trôi qua scan matching. SLAM/AMCL lấy odometry làm giá trị dự đoán và dùng scan để hiệu chỉnh.

3. Bản đồ lưới chiếm dụng và hai file `map.pgm`, `map.yaml`. `map.pgm` là ảnh xám: 0 (đen) là có vật cản, 254 (trắng) là trống, 205 là chưa biết. `map.yaml` là metadata: tên ảnh, `resolution` (m/điểm), `origin` (tọa độ góc dưới trái), ngưỡng `occupied_thresh`/`free_thresh`, `negate`.

4. Localization khác mapping thế nào? AMCL hoạt động ra sao? Mapping dựng bản đồ chưa có; localization giữ bản đồ cố định và chỉ ước lượng vị trí robot trên đó. AMCL (Adaptive Monte Carlo Localization) là bộ lọc hạt: giữ một tập hạt, mỗi hạt là một giả thuyết tư thế. Bước *dự đoán* di chuyển các hạt theo odometry cộng nhiễu; bước *cập nhật* gán trọng số mỗi hạt theo mức scan hiện tại khớp bản đồ (mô hình `likelihood_field`); bước *lấy mẫu lại* giữ các hạt trọng số cao. "Adaptive" là số hạt tự thay đổi (500–2000) theo độ bất định. Khi hạt hội tụ, sigma nhỏ (đo được khoảng 0,15 m và $8°$).

5. Vì sao phải đặt initial pose? Không có vị trí ban đầu, các hạt không biết bắt đầu ở đâu trên bản đồ; nhiều chỗ trong bản đồ trông giống nhau nên dễ hội tụ sai. Initial pose thu hẹp không gian tìm kiếm. Khi em đặt sai, AMCL ở lại tư thế lệch (xem Phần 3); có thể dùng `/reinitialize_global_localization` để rải hạt khắp bản đồ, nhưng trong lần thử của em kết quả không đáng tin (sigma khoảng 1,7–3,0 m và vị trí ước lượng lệch xa).

6. Vai trò các frame `map`, `odom`, `base_link`? `map`: khung cố định gắn với bản đồ; `odom`: khung liên tục nhưng trôi; `base_link`: gắn với robot. Nav2 lập kế hoạch trong `map`, điều khiển cục bộ trong `odom` (mượt, không nhảy khi AMCL hiệu chỉnh).

7. Nav2 lập kế hoạch và điều khiển thế nào? Xem Phần 4: `planner_server` (NavFn) tìm đường toàn cục trên global costmap; `controller_server` (DWB) chọn (v, ω) bám đường đó và tránh vật cản bằng local costmap; `bt_navigator` điều phối và kích hoạt phục hồi khi kẹt; `velocity_smoother` làm mượt lệnh trước khi gửi `/cmd_vel`.

8. Costmap và inflation để làm gì? Costmap biến bản đồ và dữ liệu cảm biến thành "chi phí đi qua mỗi ô". Inflation phồng vật cản bằng bán kính robot cộng biên an toàn, để chỉ cần coi robot là một điểm khi lập kế hoạch. Với `robot_radius`=0,175 m và `inflation_radius`=0,45 m, đường đi tránh sát tường (kết quả: cách tường $\geq 0{,}37$ m).

9. Global planner và local planner khác nhau thế nào? Global planner nhìn toàn bản đồ, ra đường đi dài hạn nhưng bỏ qua động học và vật cản mới. Local planner nhìn cửa sổ nhỏ, chạy tần số cao (20 Hz), tính lệnh vận tốc cụ thể có xét giới hạn tốc độ/gia tốc và vật cản vừa quan sát.

10. Vì sao phải chạy với `use_sim_time:=true`? Mọi node phải dùng `/clock` của Gazebo để mốc thời gian của TF, scan và odom khớp nhau; nếu lẫn giờ thật và giờ mô phỏng, TF báo "extrapolation" và các message bị loại.

11. Vì sao đặt `safety_override` và undock? Create 3 có cơ chế an toàn (hazard, reflex) và mặc định không cho robot chạy khi còn trên dock; `backup_only` cho phép Nav2 chạy mà vẫn giữ an toàn khi lùi. Undock đưa robot ra khỏi trạm sạc.

8 Kết luận, hạn chế và liên kết

8.1 Kết quả đạt được

- Phần 1: robot chạy trong Gazebo; `/scan`, `/odom`, `/cmd_vel` hoạt động (log `logs/01_topic_check.log`).
- Phần 2: dựng và lưu bản đồ `maps/map.pgm`, `maps/map.yaml`; không có tường nhân đôi.
- Phần 3: nạp bản đồ bằng `map_server`, AMCL hội tụ (sigma khoảng 0,15 m, $8°$) sau khi đặt initial pose bằng khớp scan–bản đồ.
- Phần 4: Nav2 đưa robot đến cả 4 goal ở các khu vực khác nhau mà không điều khiển tay; khoảng cách nhỏ nhất tới tường 0,37 m.

8.2 Hạn chế và những điều cần nói thẳng

- **World tự tạo, nhỏ hơn** world `maze` có sẵn; **cảm biến bị giảm tần số** bằng URDF overlay, và cần `LIBGL_ALWAYS_SOFTWARE=1`. Đề cho phép dùng môi trường có sẵn nhưng không cấm môi trường tự tạo; lý do là hiệu năng máy.
- **Bản đồ có hậu xử lý** bằng `clean_map.py` (xóa điểm lẻ bên ngoài tường); bản thô được giữ lại trong `maps/map_raw.pgm`.

- **Bản đồ được quét bằng lái tay**, đúng yêu cầu đề; node tự động bám tường em viết thử không dùng.
- **Initial pose lấy từ khớp scan–bản đồ**, nên số liệu "AMCL trùng với khớp scan" không phải kiểm chứng độc lập; chưa có vị trí thật từ Gazebo để so (topic ground truth lỗi QoS). Kiểm chứng định tính là tia LiDAR trùng tường và robot tới đúng các goal.
- **Goal 1 thất bại lần đầu** (planner không tạo được đường), thành công khi thử lại; chưa xác định chắc nguyên nhân.
- Số liệu goal ghi bằng script `send_goals.py` (cùng action `NavigateToPose`); phần đặt goal bằng công cụ Nav2 Goal của RViz2 được thể hiện trong video/ảnh.
- Lần gửi goal thứ 5 (để chụp ảnh) thất bại với lỗi TF *Transform data too old* sau khi hệ thống chạy lâu; nguyên nhân gốc chưa rõ (mục 6.3).
- Khi Wi-Fi rớt, Gazebo Transport (multicast) mất kết nối làm `/odom`, `/scan` ngừng; cần chạy lại mô phỏng. Đây là lỗi môi trường, không phải lỗi thuật toán.

8.3 Liên kết

- Video demo (Google Drive, công khai): `[LINK-VIDEO-GOOGLE-DRIVE]`
- GitHub repository (Public): `https://github.com/[GITHUB-USER]/[TÊN-REPO]`

8.4 Tài liệu tham khảo

- TurtleBot 4 Simulator: `https://github.com/turtlebot/turtlebot4_simulator`
- TurtleBot 4 User Manual: `https://turtlebot.github.io/turtlebot4-user-manual/`
- SLAM Toolbox: `https://github.com/SteveMacenski/slam_toolbox`; Nav2: `https://docs.nav2.org`